ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Facultad de Ingeniería en Electricidad y Computación

Diseño de Software

TALLER DE PATRONES CREACIONALES

Integrantes

Jonathan Alejandro Fierro Moreta

Paralelo: 101 Profesor: JURADO MOSQUERA DAVID ALONSO

Tabla de contenido

Sección A. Reporte de patrones 3
A.1 Alcance, supuestos y patrones descartados 3
A.2 Patrón Singleton: acceso único a datos 4
A.3 Patrón Factory Method: creación de productos 5
Sección B. Diagramas UML 6
B.1 Diagrama de clases con paquetes 6
B.2 Diagrama de secuencia del Singleton 7
B.3 Diagramas de secuencia del Factory Method 8
Sección C. Implementación en Java 9

Sección A. Reporte de patrones

A.1 Alcance, supuestos y patrones descartados

El sistema administra los productos de una tienda que vende artículos físicos y digitales. De todo lo que hace, el taller pide fijarnos en dos piezas: que todo el acceso a datos pase por un único punto, y que la creación de productos pueda crecer con tipos nuevos sin reescribir código que ya funciona. Por eso no se modelan pantallas, reportes ni la lógica de ventas. Para esas dos piezas identificamos el patrón Singleton (componente de acceso a datos) y el patrón Factory Method (creación de productos).

Supuestos. La base de datos central se simula en memoria dentro del propio componente de acceso, ya que el taller no exige JDBC; pasar a una conexión real solo cambiaría el interior de esa clase. La aplicación corre en una sola JVM, de modo que una instancia por proceso equivale a una instancia en todo el sistema, y varios hilos pueden pedirla a la vez. Todo producto tiene id único, nombre, precio, stock y categoría, y lo que varía entre tipos son los atributos propios: peso y dimensiones en los físicos, tamaño y formato en los digitales. Los datos del producto llegan desde la capa de interfaz como pares clave-valor, y las categorías se manejan como simples cadenas. El tipo solicitado llega como texto («FISICO», «DIGITAL») y la aplicación registra al arrancar qué creador atiende cada tipo.

Patrones descartados. Un Simple Factory con switch sobre el tipo resuelve la decisión dinámica, pero obliga a modificar la fábrica cada vez que aparece un tipo nuevo, justo lo que el enunciado quiere evitar. Abstract Factory está pensado para crear familias de objetos relacionados, y aquí cada solicitud produce un solo objeto, así que sería diseño de más. Builder sirve para armar paso a paso un objeto con muchos parámetros opcionales, pero nuestro problema es decidir qué subtipo construir, no cómo ensamblarlo. Para el acceso a datos, una clase con métodos estáticos no permitiría controlar la inicialización ni cambiar la implementación más adelante.

A.2 Patrón Singleton: acceso único a datos

Objetivo.

Garantizar una instancia de DatabaseManager por proceso para centralizar las operaciones de consulta y actualización de la base de datos.

Motivación.

Diversos módulos de la tienda necesitan consultar inventario, precios y categorías. Crear administradores independientes puede dispersar la configuración y duplicar recursos. Singleton proporciona un punto común de acceso.

Consecuencias.

Ventajas: acceso consistente, inicialización diferida y control centralizado. Limitaciones: estado global, pruebas más difíciles y posible cuello de botella al compartir una conexión. No asegura una única instancia entre procesos distribuidos.

Relación con SOLID.

SRP: el administrador concentra la responsabilidad de acceso a datos; OCP: su API evita modificar consumidores cuando cambia la implementación interna; DIP: en una solución mayor es preferible que los servicios dependan de una interfaz de repositorio, porque invocar directamente el Singleton crea acoplamiento. Singleton no garantiza SOLID por sí solo.

A.3 Patrón Factory Method: creación de productos

Objetivo.

Separar la creación de productos del código cliente, delegando la instancia concreta a una subclase de ProductFactory seleccionada en tiempo de ejecución.

Motivación.

Un producto físico exige peso y dimensiones, mientras uno digital necesita tamaño y formato. Evitar condicionales extensos permite agregar nuevos tipos sin modificar la selección existente.

Product es el producto abstracto y PhysicalProduct / DigitalProduct son productos concretos. ProductFactory define el método de fábrica create; PhysicalProductFactory y DigitalProductFactory lo sobrescriben. ProductFactoryRegistry localiza la fábrica por tipo y llama al método polimórfico. El registro es un mecanismo auxiliar de selección, no otro patrón GoF independiente.

Consecuencias.

Ventajas: expansión por nuevas clases, creación encapsulada y menor dependencia del cliente respecto a tipos concretos. Costos: más clases, necesidad de registrar cada fábrica y validación de atributos específicos en tiempo de ejecución.

Relación con SOLID.

OCP: se añaden productos y fábricas sin editar ProductFactoryRegistry; LSP: cualquier fábrica concreta sustituye a ProductFactory; DIP: el registro utiliza la abstracción ProductFactory y retorna Product; SRP: cada fábrica solo construye su tipo. ISP no es el foco porque el contrato create es pequeño.

Colaboración entre patrones. CreadorProducto obtiene el GestorBaseDatos con getInstancia() para persistir lo que fabrica, así que los dos patrones se conectan en ese único punto.

Sección B. Diagramas UML

B.1 Diagrama de clases con paquetes

Cada patrón está en su propio paquete: datos contiene el Singleton y productos contiene el Factory Method; el paquete app solo tiene el cliente de prueba. Se muestran la visibilidad de atributos y métodos (+ público, - privado, # protegido), los miembros estáticos subrayados, los abstractos en cursiva, las herencias, las dependencias de creación y las multiplicidades.

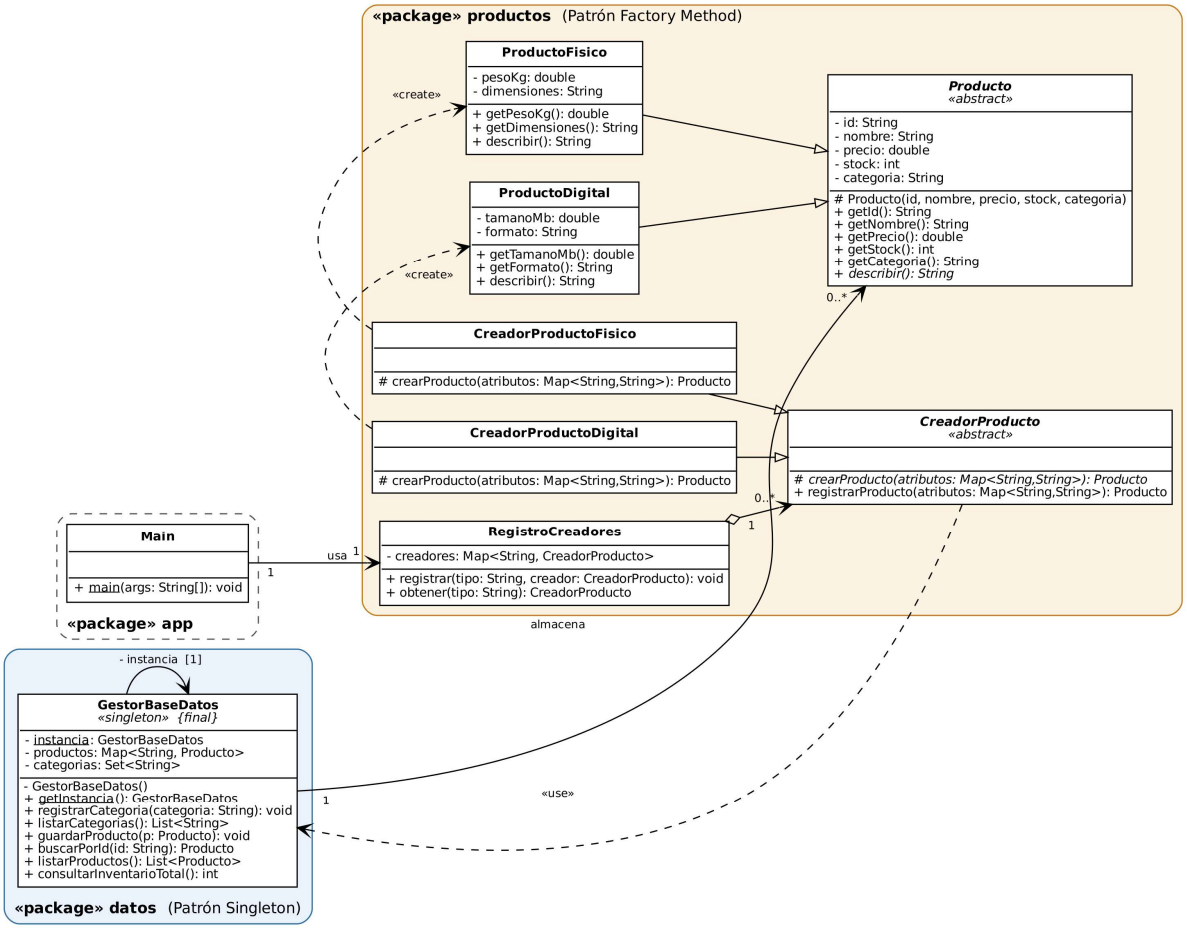


Figura 1. Diagrama de clases del sistema, con paquetes separados por patrón.

B.2 Diagrama de secuencia del Singleton

El diagrama muestra el mecanismo del patrón. En la primera llamada a getInstancia() se carga la clase interna Holder y se construye la única instancia. Cuando otro cliente la pide después, recibe exactamente la misma y no se vuelve a construir nada, por lo que lo que guarda uno es visible para el otro.

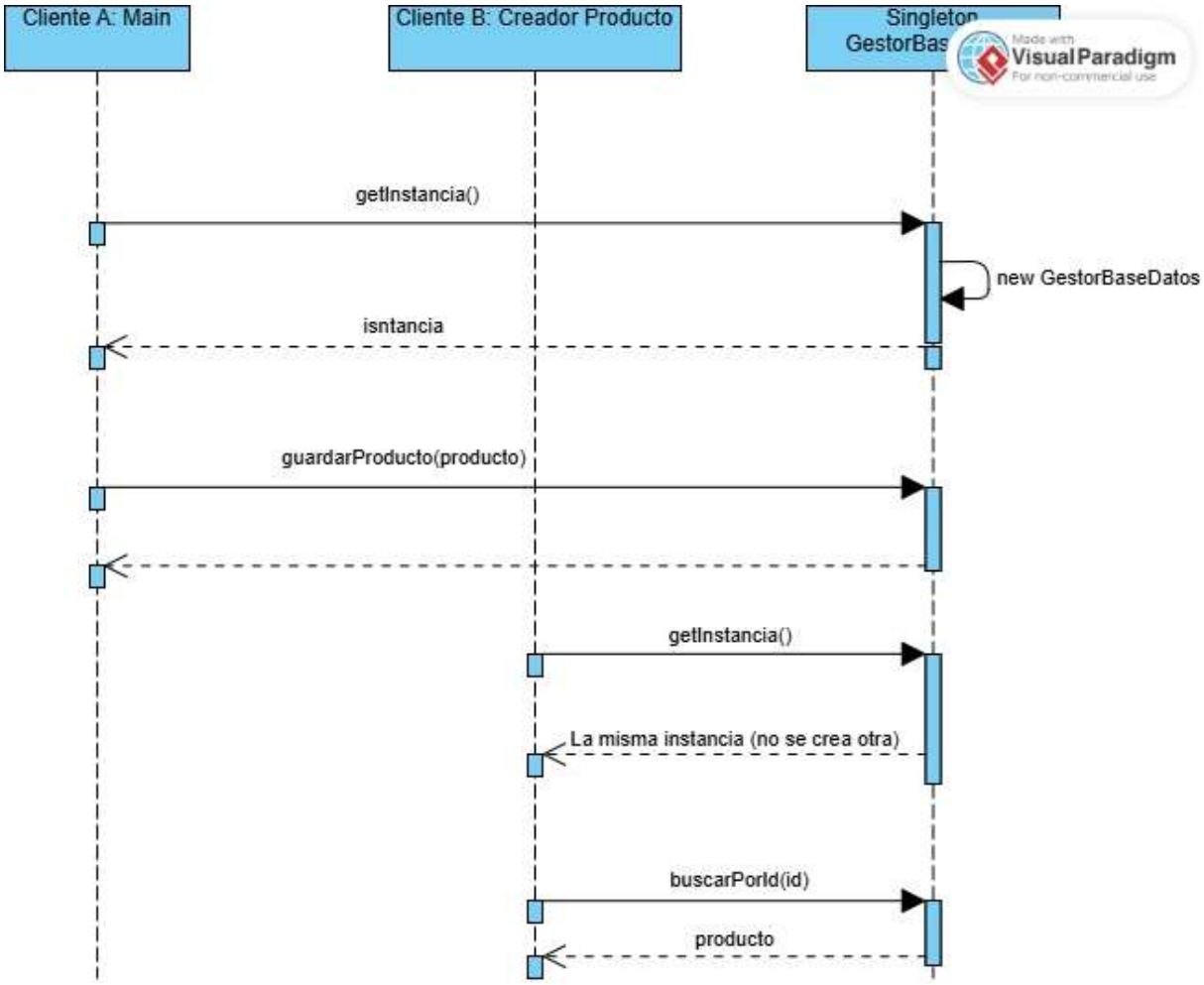


Figura 2. Secuencia del Singleton GestorBaseDatos con dos clientes.

B.3 Diagramas de secuencia del Factory Method

En ambos casos Main pide el creador al RegistroCreadores según el tipo y llama a registrarProducto. El creador concreto ejecuta el método fábrica crearProducto, que decide el subtipo, y luego guarda el producto usando la instancia única del gestor. Lo único que cambia entre las dos figuras es el creador y el producto que se construye.

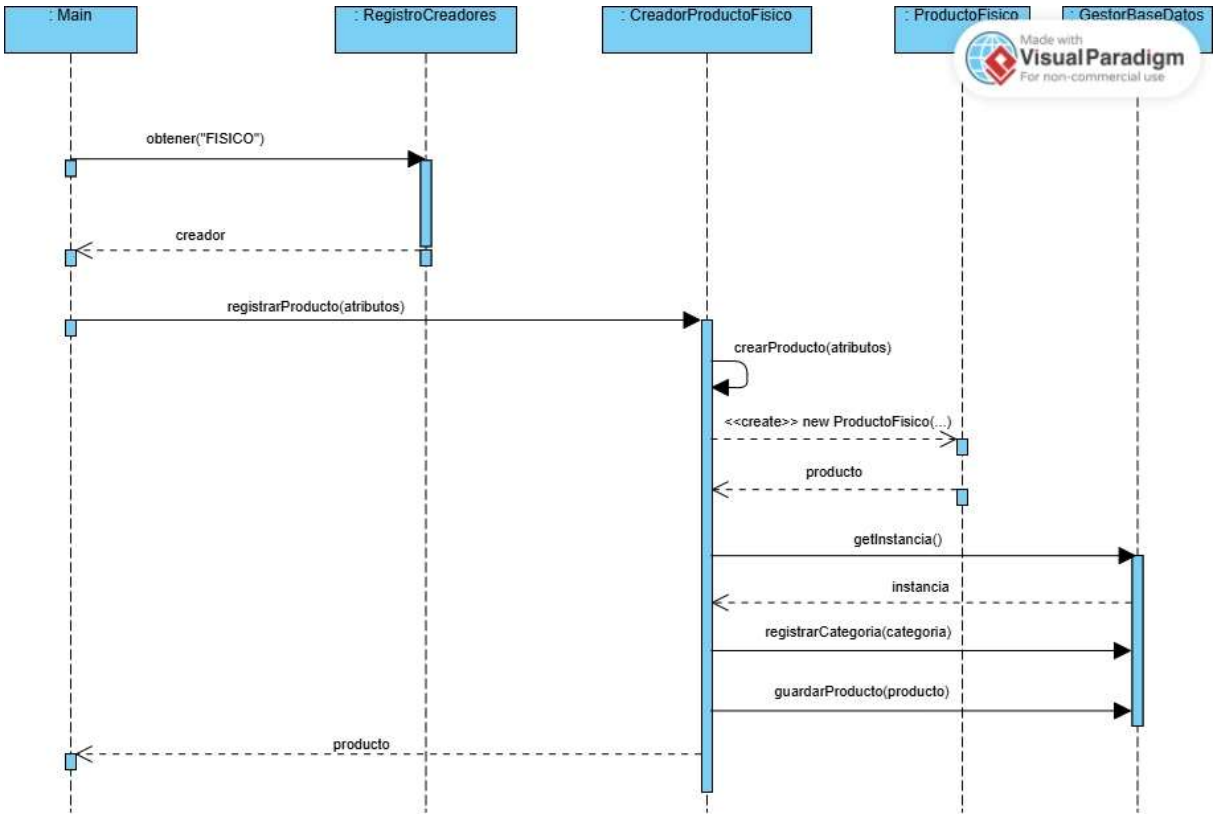


Figura 3. Creación de un producto físico.

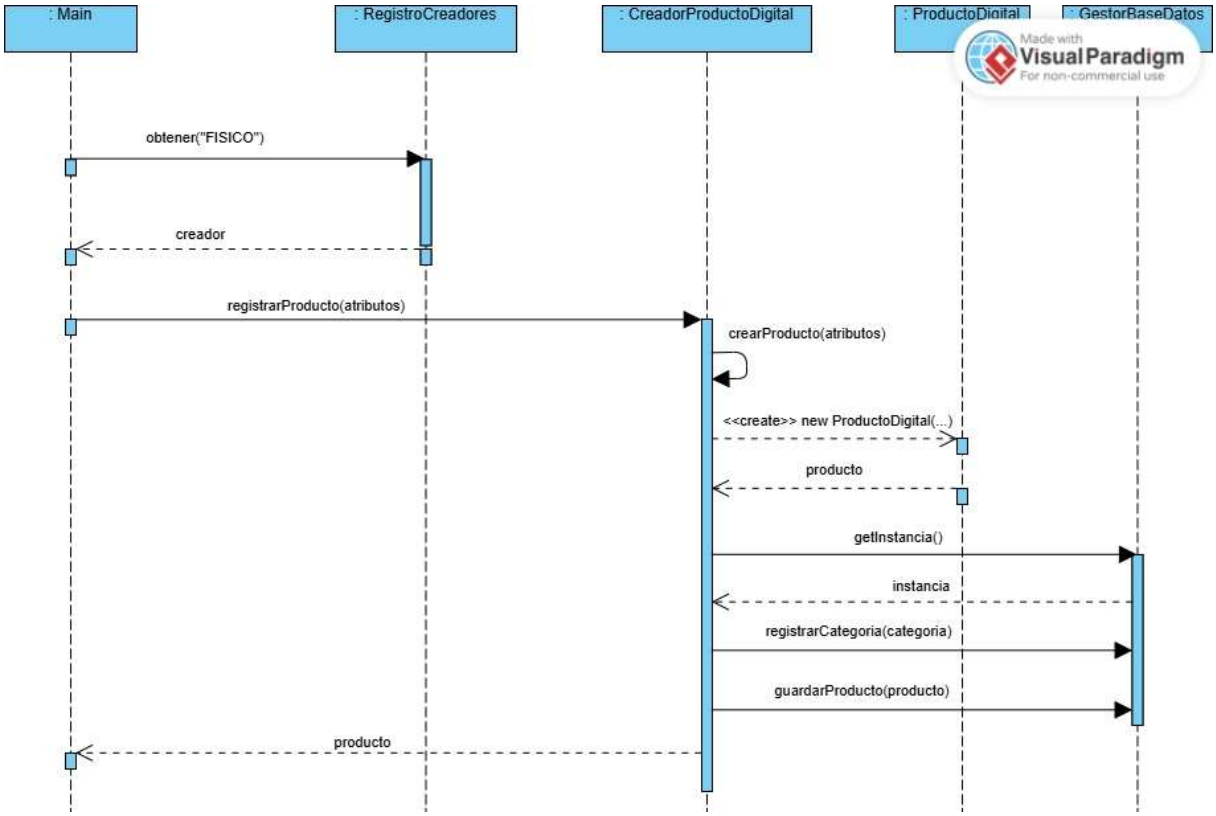


Figura 4. Creación de un producto digital.

Sección C. Implementación en Java

El código mínimo está en la carpeta src, organizado en los mismos paquetes del diagrama: datos (GestorBaseDatos), productos (Producto, ProductoFisico, ProductoDigital, CreadorProducto, CreadorProductoFisico, CreadorProductoDigital y RegistroCreadores) y app (Main). Se compila con javac -d out y todos los .java, y se ejecuta con java -cp out app.Main. A continuación se muestran los fragmentos clave.

Singleton. Constructor privado y acceso mediante el Holder:

```
public final class GestorBaseDatos {
    private GestorBaseDatos() { /* conexión simulada */ }
    private static final class Holder {
        private static final GestorBaseDatos INSTANCIA = new GestorBaseDatos();
    }
    public static GestorBaseDatos getInstancia() { return Holder.INSTANCIA; }
}
```

Factory Method. El creador abstracto fija el flujo y la subclase decide el subtipo:

```
public abstract class CreadorProducto {
    protected abstract Producto crearProducto(Map<String, String> atributos);
    public Producto registrarProducto(Map<String, String> atributos) {
        Producto producto = crearProducto(atributos);          // método fábrica
        GestorBaseDatos bd = GestorBaseDatos.getInstancia();   // Singleton
        bd.registrarCategoria(producto.getCategoria());
        bd.guardarProducto(producto);
        return producto;
    }
}
public class CreadorProductoFisico extends CreadorProducto {
    @Override
    protected Producto crearProducto(Map<String, String> a) {
        return new ProductoFisico(texto(a, "id"), texto(a, "nombre"), decimal(a, "precio"),
                entero(a, "stock"), texto(a, "categoria"),
                decimal(a, "peso"), texto(a, "dimensiones"));
    }
}
```

Selección dinámica. RegistroCreadores guarda un mapa de tipo a creador y lo consulta sin condicionales; para un tipo nuevo basta registrar su creador, como hace Main con «SERVICIO»:

```
RegistroCreadores registro = new RegistroCreadores();
registro.registrar("FISICO", new CreadorProductoFisico());
registro.registrar("DIGITAL", new CreadorProductoDigital());
registro.registrar("SERVICIO", new CreadorProductoServicio());  // tipo nuevo, sin tocar lo
existente
Producto p = registro.obtener("digital").registrarProducto(atributos);
```

Salida de la ejecución. Se ve que la conexión se crea una sola vez, que los tres tipos se fabrican correctamente y que las dos referencias al gestor son el mismo objeto:

```
[DB] Conexión creada
FISICO  [P001] Laptop | $850.00 | stock=10 | cat=Electrónica | peso=1.8kg | dim=35x24x2 cm
DIGITAL [P002] Libro de Java | $15.50 | stock=999 | cat=Libros | tamaño=12.4MB | formato=PDF
SERVICIO [P003] Soporte técnico | $40.00 | stock=0 | cat=Servicios | horas=2
¿Misma instancia? true
Categorías: [Electrónica, Libros, Servicios]
Inventario total: 1009
Buscar P002: Libro de Java
Error esperado: Tipo de producto no soportado: HOLOGRAMA
```